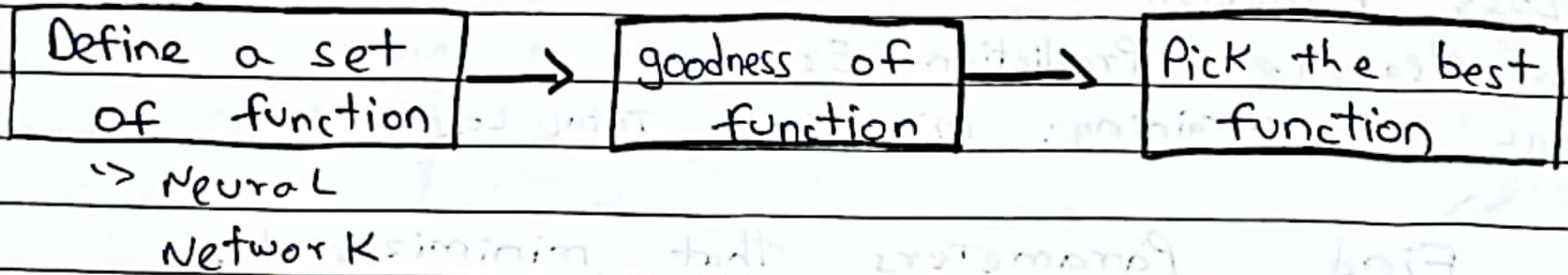
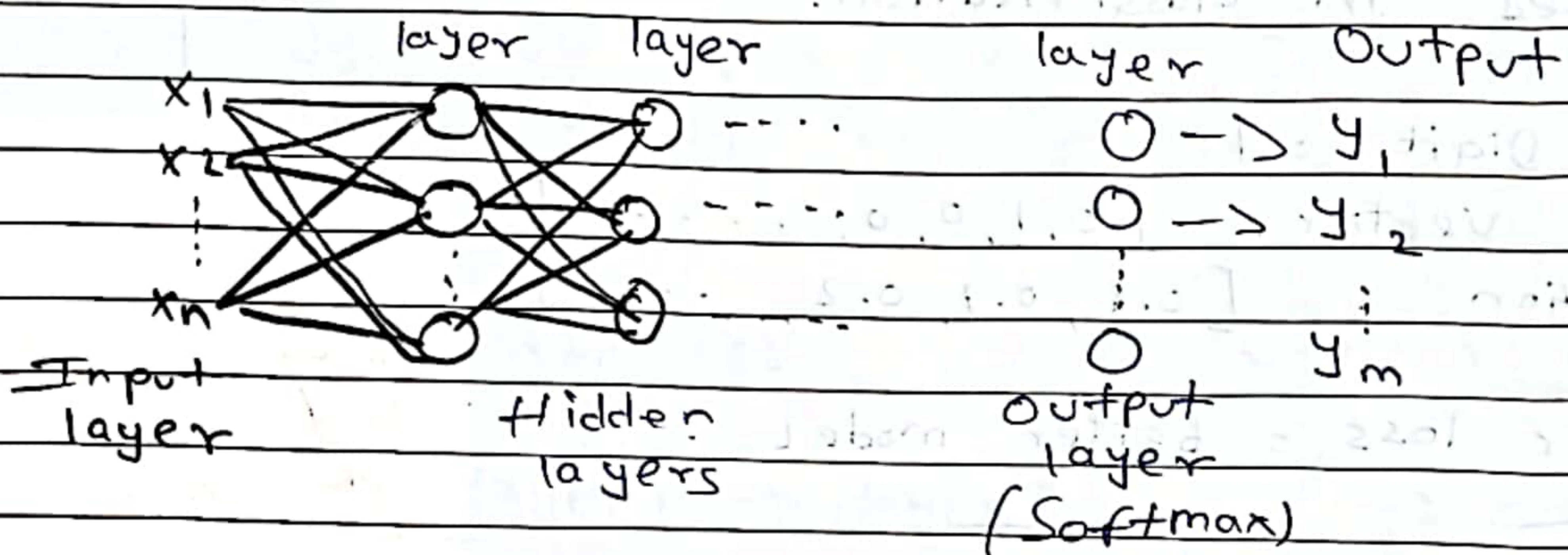


Deep-learning (Coursera)



** Fully connected Feedforward Network.



$$a = wx + b$$

Neural network = stack of matrix operations.

* More layers, better feature extraction

Edges $\rightarrow$ Shapes $\rightarrow$ Objects $\rightarrow$ Classes.

ex: Digit recognition $\rightarrow$

Input $\rightarrow$ 16×16 Image = 256 Features

Output $\rightarrow$ 10 Classes (0-9)

ex: output $\rightarrow$ $[0.1, 0.7, \dots]$

each value = highest Probability of a digit
Highest value among it = Prediction

Loss Function

→ Measure of Prediction Error

Goal of Training: minimize Total Loss

↘
 Find Parameters that minimizes loss

* Cross-Entropy Loss

↳ Used in Classification.

ex: Digit 1 = 1 - 0

Target Vector: $[0, 1, 0, 0, \dots]$

Prediction: $[0.1, 0.7, 0.2, \dots]$

lower loss = better model

Pipeline: -

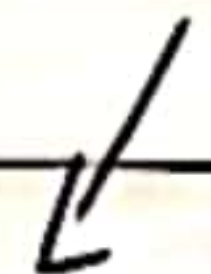
① Define Model

Layers, Neurons, Architecture

② Define loss

③ Optimize Parameters

using Gradient descent, Back propagation.



→ Update weights to Reduce loss

$$\text{new} = \text{old} - \eta \times \nabla$$

Meaning: Gradient shows direction of Increase.

Back Propagation:->

-> Efficient Gradient Computation

-> Use Chain Rule.

Compute: $\frac{dLoss}{dw}$

ex: In Neural Network:-

$$y = f(g(h(x)))$$

$$\frac{dy}{dx} = \frac{dy}{df} \times \frac{df}{dg} \times \frac{dg}{dh} \times \frac{dh}{dx}$$

a) Forward Pass

-> Compute Predictions

Input -> layer 1 -> layer 2 -> ... -> output

$$z = wx + b$$

$$a = \text{activation}(z)$$

-> It stores Intermediate values needed for Backward Pass.

b) Backward Pass

-> Compute gradients.

Loss -> Output layer -> Hidden layers -> Input

Compute: $\frac{dL}{dw}$ -> Use Chain Rule:->

X

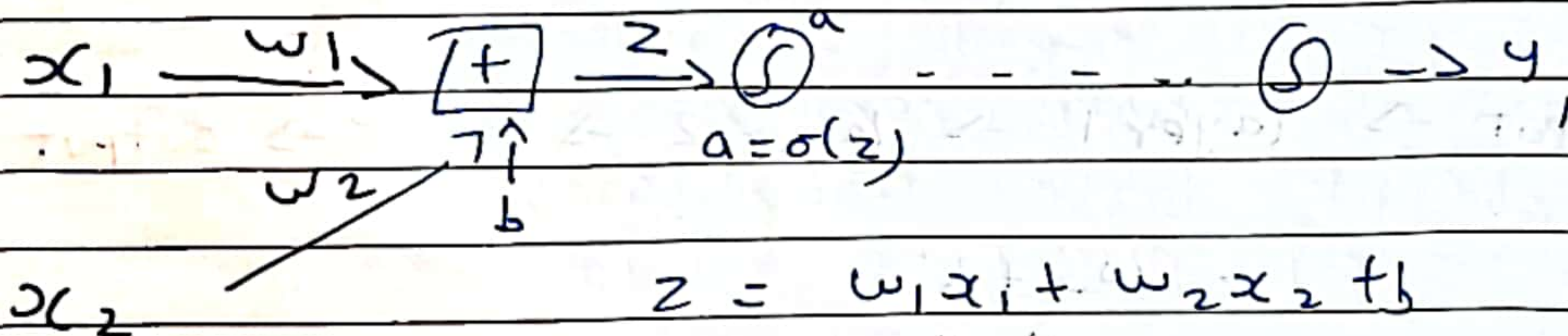
* Backward Pass cases:

a) output layer.

$$\frac{dL}{dz} \rightarrow \frac{dL}{dw} = \frac{dL}{dz} \times \frac{dz}{dw}$$

b) Hidden layers.

Hidden gradient = next layer gradient $\times$ local derivative



Update weights

$$w = w - \eta \frac{dL}{dw}$$

Repeat until convergence.

CNN (Convolutional Neural Networks)

* CNN for Images $\rightarrow$ features

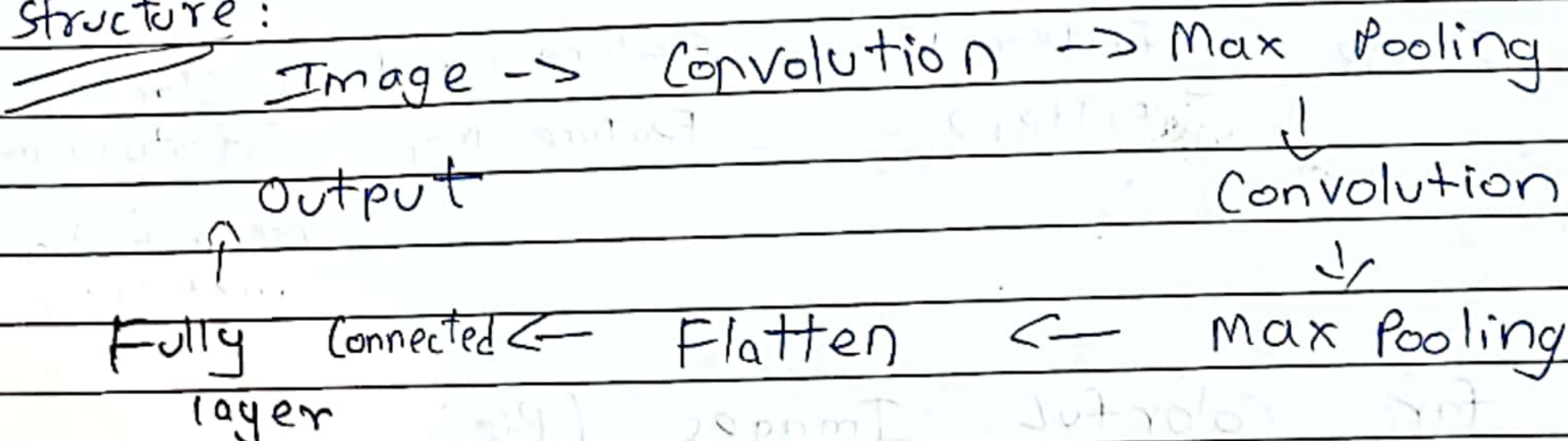
1. Small patterns form the whole Image
2. Some Patterns appear in diff. regions.

(Parameter Sharing)

→ (pooling)

3. Subsampling does not change object Identity

Structure:



Convolution - How filters work.

① Each filter detects a small local pattern

(3x3) matrix

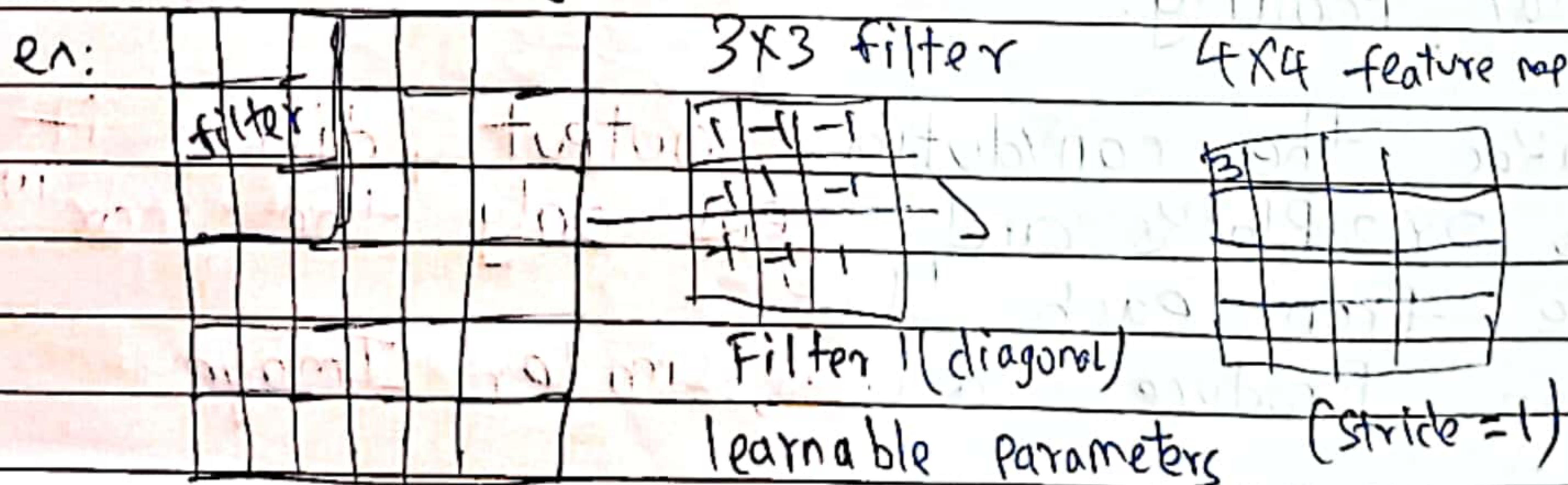
ex: Filter 1 (diagonal) Filter 2 (vertical)

1 -1 -1 / -1 -1 / -1 -1 -1 -1 / -1 -1 / -1 -1

Filter 1 responds to diagonal patterns

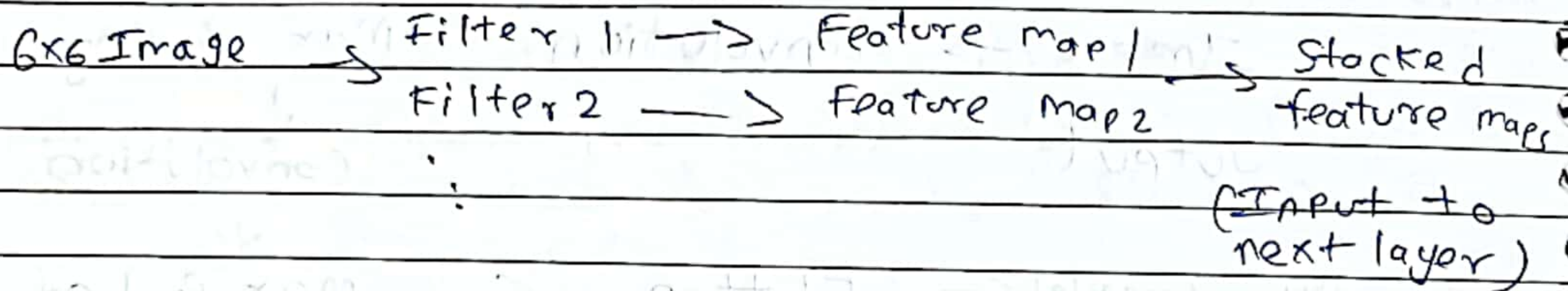
Filter 2 responds to vertical patterns

② The same filter scans the whole Image with a fixed step called the stride.
 6x6 Image



dot Product → scalar

3- Each filter produces one feature map.



4. for Colorful Images (RGB)
 Each channel has 6×6 matrix.
 each filter is 3×3

* Convolution vs fully connected.

→ In fully connected layer every Input Pixel connects to every neuron, In convolution each output connects to only 9 Pixels under 3×3 filter $\therefore$ fewer Parameter.

→ The same filter is reused at every position across the Image. (Weight sharing)

Max Pooling.

→ Takes the convolution output, divides it into 2×2 Blocks and keeps only the max^m value from each block.

→ Then Produces a new, smaller Image

example: -

Filter 1 (diagonal Pattern)

Block $[3, -1, -3, 1] \rightarrow \max = 3$

Block $[-3, -1, 0, -3] \rightarrow \max = 0$

Block $[-3, -3, 3, -2] \rightarrow \max = 3$

Block $[0, 1, -2, -1] \rightarrow \max = 1$

Filter 2 (vertical pattern)

Block $[-1, -1, -1, -1] \rightarrow \max = -1$

Block $[-1, -1, -2, 1] \rightarrow \max = 1$

Block $[-1, -1, -2, 1] \rightarrow \max = 1$

Block $[-1, 0, -4, 3] \rightarrow \max = 3$

After Conv + max Pooling.

$[-1, 1 | 0, 3]$

* Flatten

After Max-Pooling Output is a 3-D Tensor.
Flatten converts it into 1-D vector for the fully connected layer.

ex: 2×2 Feature map with 2 channels

$\rightarrow 8$ values

Output of Flatten: $[3, 0, 1, \dots]$

Then feed into fully connected feed forward network.

CNN in Keras.

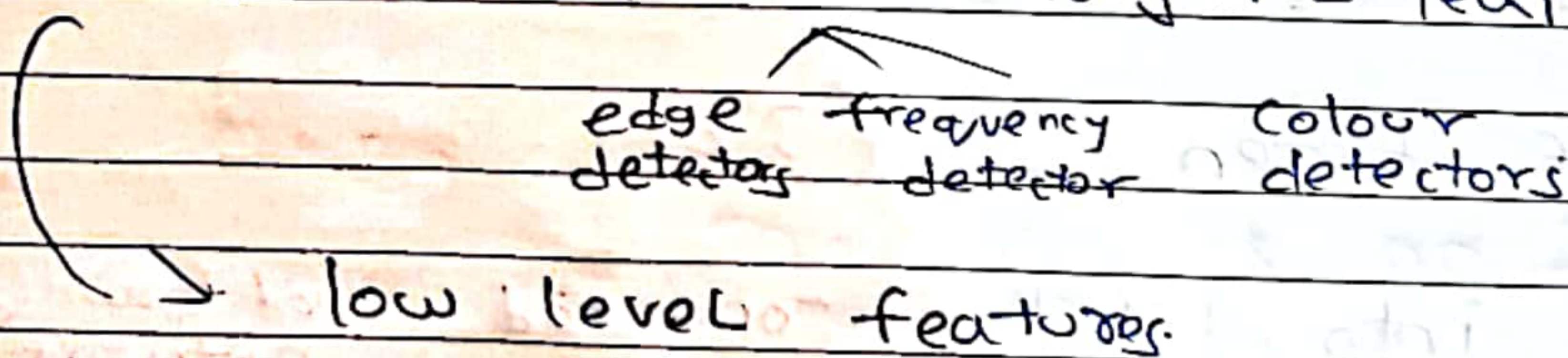
$\rightarrow$ Network Structure (adding Conv 2D, Max Pooling 2D, Flatten layers.)

Input = (28, 28, 1)

layer	Keras Code	output shape	Params
* Input	Input_shape = (28, 28, 1)	1x28x28	
* Conv2D (25 filters, 3x3)	Convolution 2D (25, 3, 3..)	25x26x26	9
* Max Pooling 2D (2x2)	Max Pooling 2D ((2, 2))	25x13x13	-
* Conv 2D (50 filter, 3x3)	Convolution 2D (50, 3, 3)	50x11x11	225
* Max Pooling 2D (2x2)	Max Pooling 2D ((2, 2))	50x5x5	-
* Flatten	Flatten()	1250	-
* Dense (100)	Dense (output_dim=100) + ReLU	100	-
* Dense (10)	Dense (output_dim=10) + softmax	10 (classes)	-

What does CNN Learn?

* First Convolution layer - learned filters



Pixels → edges → shapes → Parts → Objects.

* To find what a neuron learns we must find images that produces maximum activation for it.

→ Higher layers detect semantic concepts

* Gradient Ascent Approach

→ After Conv + MaxPool, filter K produces a 11×11 feature map.

a_{ij}^K

Total activation of filter k : $a^K = \sum_{i=1}^H \sum_{j=1}^H a_{ij}^K$

→ We want an Input Image that maximizes $x^* = \arg \max a^K$

→ we freeze weights & update Input Pixels.

Steps:

- 1] Initialize Image
- 2] Forward Pass → compute activation
- 3] Compute da^K

4] Update Pixels.

$$x = x + \eta da^K$$

5] Repeat until Converge.

→ Gradient shows what pixel changes increase neuron response.

* Fully connected neurons → Combination of Patterns ∴ Visualization becomes more abstract deeper in network.

Apply Gradient Ascent.

* for output layer

→ Result without regularisation would lead to random noise, no human meaning, fooling by unrealistic inputs

Adding Regularisation → still abstract but closer to actual

$$x^* = \arg \max_{x \in \mathbb{R}^n} (y^T - \sum_{i=1}^n |x_i|)$$

→ minimise Pixel magnitude

→ maximise class score.

Applications of DL

1. * Deep style: →

Content Image → Photo structure
Style Image → Painting (texture/style)

3 CNN Passes.

→ Content Image

→ Style Image

→ Generated Image → (Optimized to match both)

2. AlphaGo

→ Go Board Treated as an Image

Black Stone → 1

White stone → -1

empty → 0

→ Trained through Supervised learning from expert games:

Loss: (cross entropy).

→ CNN works well for go - due to the following Reasons:-

a) local Patterns

b) Translation Invariance (weight sharing)

* Max Pooling not used because here position matters as Pooling keeps object Identity.

∴ CNN components are not mandatory, but Architecture must match problem structure.

3. Speech Recognition

4. Text Processing.

Testing vs Training Performance.

Training good vs Testing Bad → overfitting

Training Bad vs Testing Bad → Underfitting
Poorly trained

Overfitting → model memorizes the training data but fails to generalize

Fixing Overfitting! →

A) Regularization: It adds Penalty to the loss function to prevent weights from becoming too large.

$$\|w\|^2 = w_1^2 + w_2^2 + \dots$$

$$w^{t+1} = w^t - \eta \frac{\partial L}{\partial w} = \eta \lambda \operatorname{sgn}(w)$$

L1: (Lasso) $\rightarrow$ leads to sparse weights.

Loss = Original Loss + $\lambda \|w\|$

$$L'(w) = L(w) + \frac{\lambda}{2} \|w\|^2$$

L2: (Ridge) $\rightarrow$ Prevents any single weight from dominating.

Loss = Original Loss + λw^2

$$L'(w) = L(w) + \frac{\lambda}{2} \|w\|^2 \rightarrow w^{t+1} = (1 - \eta \lambda) w^t - \eta \frac{\partial L}{\partial w}$$

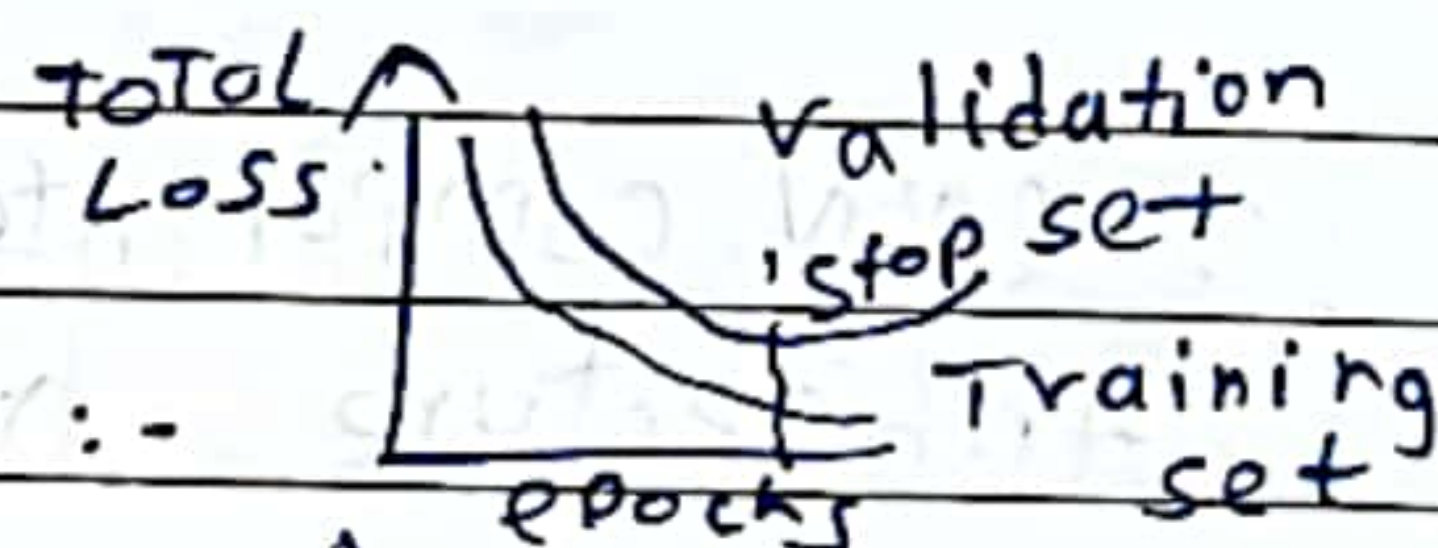
b) Dropout :-

Randomly turns off a Percentage of Neurons during training.

c) Early Stopping :-

$\rightarrow$ Monitor the validation error :-

$\rightarrow$ When training error keeps dropping but validation error starts to rise, Stop training.



d) Better Activation Functions like ReLU
Use Advanced optimizers like Adam, RMSProp
Better Initialisations

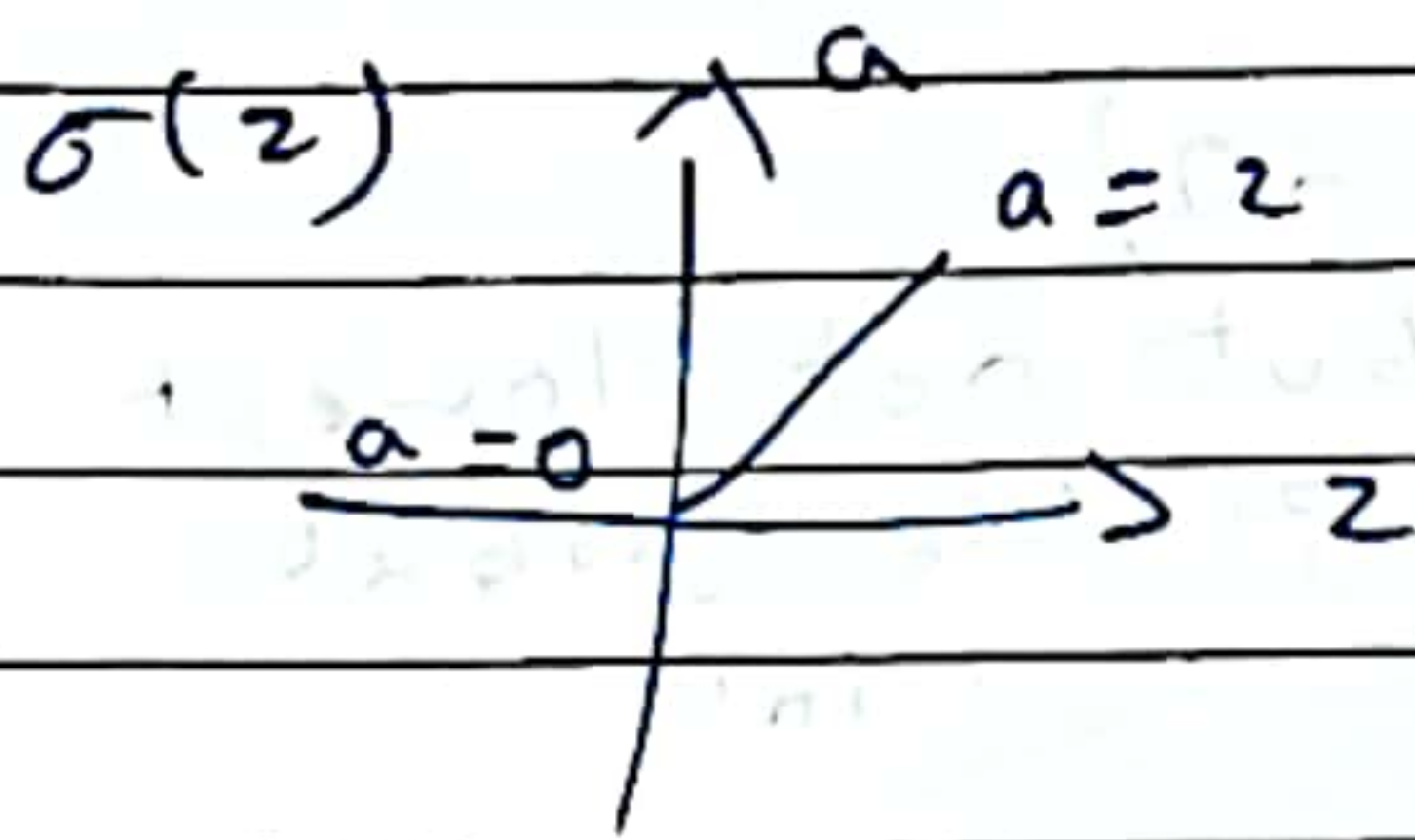
Vanishing gradient Problem :-

During Back-propagation, errors are passed backward through the network using chain Rule. Because derivatives of activation functions are less than 1, multiplying them repeatedly causes gradient to shrink automatically.

$\therefore$ Early layers stop learning and network fails to capture foundational features.

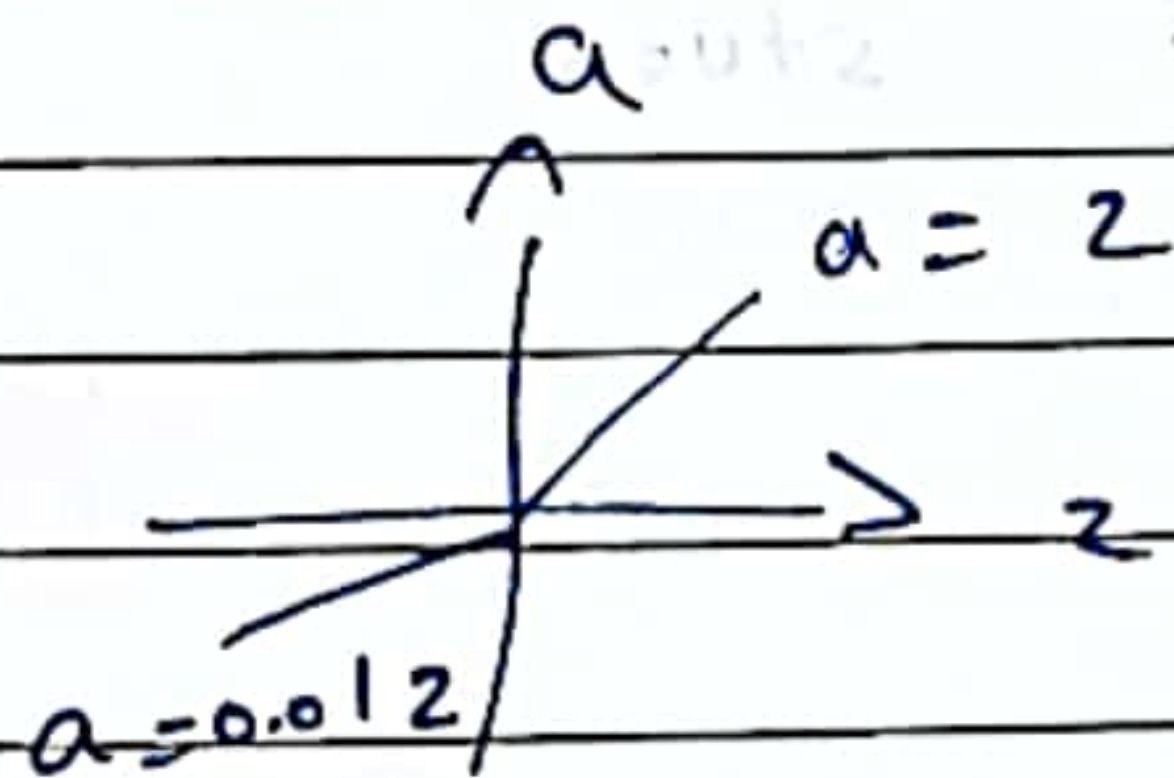
→ solves vanishing gradient problem

Activation functions : RELU → fast computation



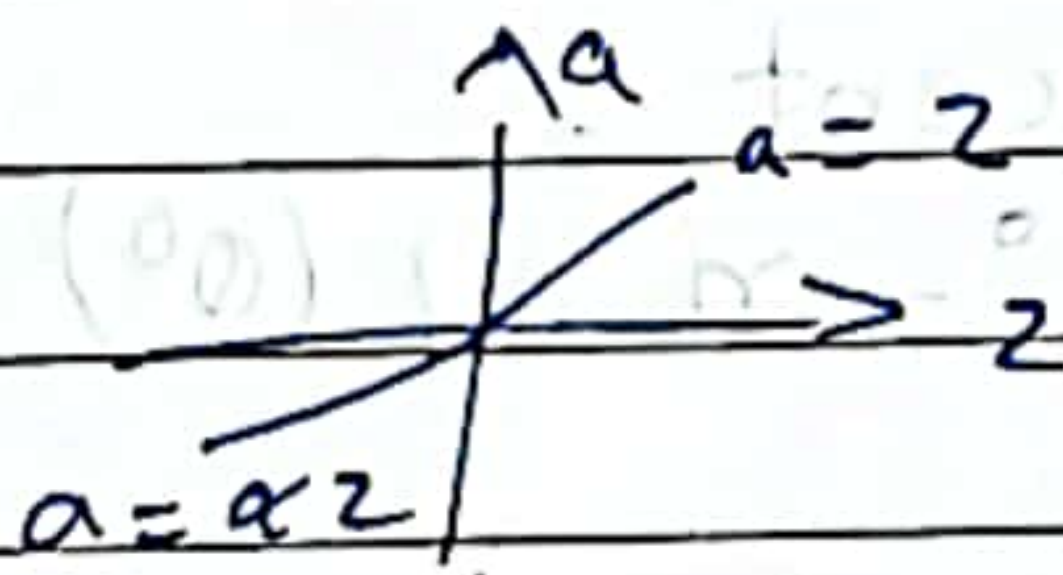
$$f(x) = \max(0, z)$$

Leaky-ReLU



$$f(x) = \begin{cases} z & x > 0 \\ 0.01z & x \leq 0 \end{cases}$$

Parametric-ReLU



$$f(x) = \begin{cases} z & x > 0 \\ \alpha z & x \leq 0 \end{cases}$$

* Maxout networks.

→ It is a generalization of RELU, In this maximum of multiple linear functions is taken.

Output = $\max(w_1 x + b_1, w_2 x + b_2)$ ex: $T = \max(10x + 10, 20x + 10)$

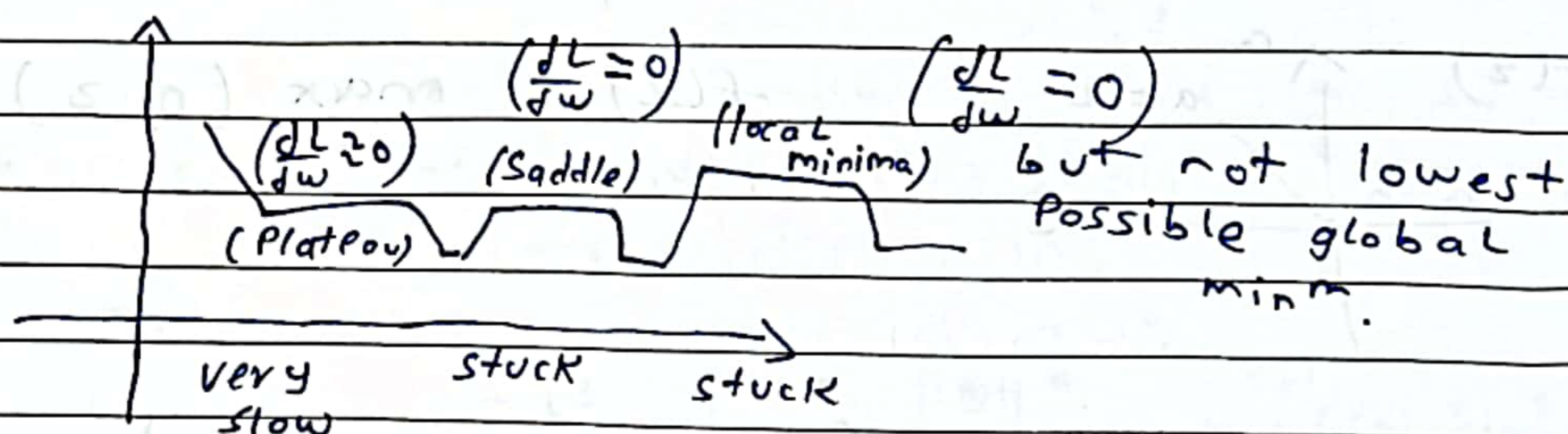
($\rightarrow$ $w_1 = 1, b_1 = 0, w_2 = 0, b_2 = 0$)

($\rightarrow$ Replicates RELU: $\max(x, 0)$)

→ It is flexible & can approximate any Convex function.

→ Only Path that Produces maximum value is updated.

** Common optimization road blocks: ->



** OPTIMIZERS

a) Vanilla gradient descent :-

$$\theta' = \theta^0 - \eta \nabla L(\theta^0)$$

b) Adaptive learning rates:

(i) Adagrad :-

- > It adapts learning rate for each parameter individually
- > It applies smaller learning rate to prevent overshooting. (small $\epsilon \downarrow$, large gradients)
- > It applies larger learning rate to accelerate learning. (large $\epsilon \uparrow$, small gradients)

$$w^{T+1} \leftarrow w^T - \frac{\eta}{\sqrt{\sum_{i=0}^T (g_i)^2}} g^T$$

(ii) RMSProp: -

Problem with Adagrad: $\rightarrow$ It accumulates all past squared gradients in denominator & learning rate shrinks to zero, stopping training entirely.

RMSProp: exponential decaying average: $\rightarrow$

$$\omega^{T+1} \leftarrow \omega^T - \frac{\eta}{\sigma^T} g^T$$

$$\sigma^T = \sqrt{\alpha (\sigma^{T-1})^2 + (1-\alpha) (g^T)^2}$$

c) Momentum.

$\rightarrow$ It is designed to blast through plateaus & escape local minima.

Intuition: Rolling a heavy ball down the hill, its movement is determined by slope & speed (gradient & Inertia)

$$\theta^t = \theta^{t-1} + v^t$$

$$v^t = \lambda v^{t-1} - \eta \nabla L(\theta^{t-1})$$

x -

Dropout Regularization :-

→ During training neurons are randomly dropped with probability p . During Testing network is connected but weights are scaled by $(1-p)$ to compensate.

Q. Consider a Neuron z receiving two inputs x_1 & x_2 with weights w_1 & w_2 $p=0.5$.

⇒ 2^m equations for z ⇒ $2^2 = 4$

Both active : $z_1 = w_1 x_1 + w_2 x_2$

Similarly : $z_2 = w_1 x_1$

$z_3 = w_2 x_2$ $z_4 = 0$

Ensemble average $E(z) = \frac{z_1 + z_2 + z_3 + z_4}{4}$
(Testing phase)

$$E(z) = \frac{2w_1 x_1 + 2w_2 x_2}{4}$$

$$E(z) = \frac{w_1 x_1}{2} + \frac{w_2 x_2}{2}$$

X

RNN (Recurrent neural networks)
 → Designed for sequential data

Input + Previous memory → output

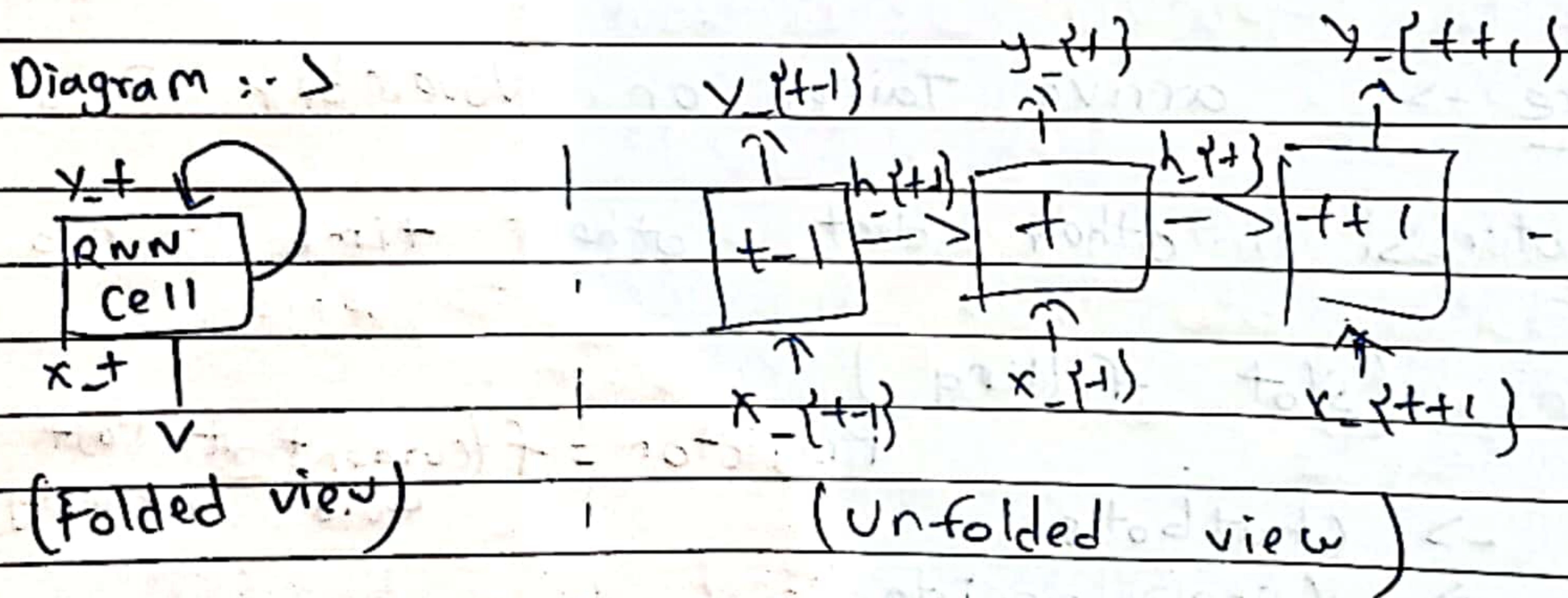
$$h_t = f(w_x * x_t + w_h * h_{t-1})$$

$$y_t = g(h_t)$$

x_t → current Input
 h_{t-1} → Previous memory.

→ Memory can be considered as another input
 → Output changes based on Previous Inputs

Sequence & memory both matter.



* Same weights (w_x, w_h) used repeatedly,
 Network is reused again & again reducing
 Parameter & allow sequence learning.

* Variable length Input.

→ hidden state: (Memory)

Stores Important features only *
Current Input + Past memory *

ex: Same word can have different outputs depending on hidden state

arrive Taipei → Destination

leave Taipei → Departure

→ Sequence labelling:

Sentence → arrive Taipei on November 2nd

output: → other dest other time time

(Slot filling)

Predictor = $f(\text{current word} + \text{Past memory})$

Uses → chatbots
→ voice-assistants.

* Word Representation (Input → RNN)

(A) One-hot encoding

ex: apple = [1 0 0 0 0]
dog = [0 0 0 1 0]

Problem: High dimension, No similarity

(B) Word - hashing

Breaks word into sub-parts (n-grams)

ex: apple $\rightarrow$ a-p-p, p-l-e, etc..

Pros: Compact, handles unknown words.

Modern: embeddings (word $\rightarrow$ vec)

* Types of RNNs: $\rightarrow$

1) Elman network.

$$h_t = f(x_t + h_{t-1})$$

Most commonly used

2) Jordan network.

Instead of hidden state, $\rightarrow$
output is fed back.

** Bi-directional RNNs: $\rightarrow$

$\rightarrow$ Sequence in two directions

Forward $\rightarrow$ Past $\rightarrow$ Future

Backward $\rightarrow$ Future $\rightarrow$ Past

Q. Sentence: She is Playing bass
= Predict meaning of bass.

Class A $\rightarrow$ musical Instrument, class B $\rightarrow$ fish

M	T	W	T	F	S	S
Page No.:						YOUVA
Date:						

$\Rightarrow$ Normal RNN ($L \rightarrow R$)

Model only sees she is playing
Prediction $\rightarrow$ uncertain.

Bidirectional RNN ($L \rightarrow R$ & $R \rightarrow L$) (Future context)

At word bass, left she is playing
Right Guitar

$\therefore$ bass = Musical Instrument.

* Deep RNN: $\rightarrow$
Stack Multiple RNN layers

Input $\rightarrow$ RNN $\rightarrow$ RNN $\rightarrow$ Output

$\rightarrow$ learns more complex patterns
 $\rightarrow$ Better feature extraction

limitations (RNN):

$\rightarrow$ Forgets old info in long sequences
 $\rightarrow$ Vanishing gradient Problem.

* Training (RNN) Done through

Backpropagation through time (BPTT)

(Gradient clipping used)

$T_5 \leftarrow T_4 \leftarrow T_3 \leftarrow T_2 \leftarrow T_1$

$\rightarrow$ error flows back through all the time steps

$\rightarrow$ same weights updated multiple times

Problems with RNNs :->

-> Vanishing gradients Problem :
explodes

-> long-Range dependency Issue:->
They cannot connect info separated by more than 10-20 steps

-> each layer remain undertrained, while later layers overfit to recency.

↓
Gated solutions

✓

LSTM :- Long Short-Term Memory:-

-> Maintains separate memory -> cell state (c)
Controls flow using 3 gates.

1. Forget Gate:->

It decides what to remove from the past memory.

$$f_t = \sigma(w_f[h_{t-1}, x_t] + b_f)$$

2. Input Gate:->

Controls what new info to add.

$$\text{Gate: } i_t = \sigma(w_i[h_{t-1}, x_t] + b_i)$$

Candidate memory:->

$$\tilde{c}_t = \tanh(w_c[h_{t-1}, x_t] + b_c)$$

Generates new candidate memory.

3. Cell status update.

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t$$

4. Output gate.

$$o_t = \sigma(w_o[h_{t-1}, x_t] + b_o)$$

final hidden state:->

$$h_t = o_t \cdot \tanh(c_t)$$

GRU :- Gated Recurrent Unit

-> No separate cell state

-> Uses hidden state only & has fewer parameters.

1. Update Gate

Controls how much past to keep.

$$z_t = \sigma(w_z[h_{t-1}, x_t] + b_z)$$

2. Reset Gate:-

Controls how much Past to forget while computing new state.

$$r_t = \sigma(w_r[h_{t-1}, x_t] + b_r)$$

3. Candidate hidden state

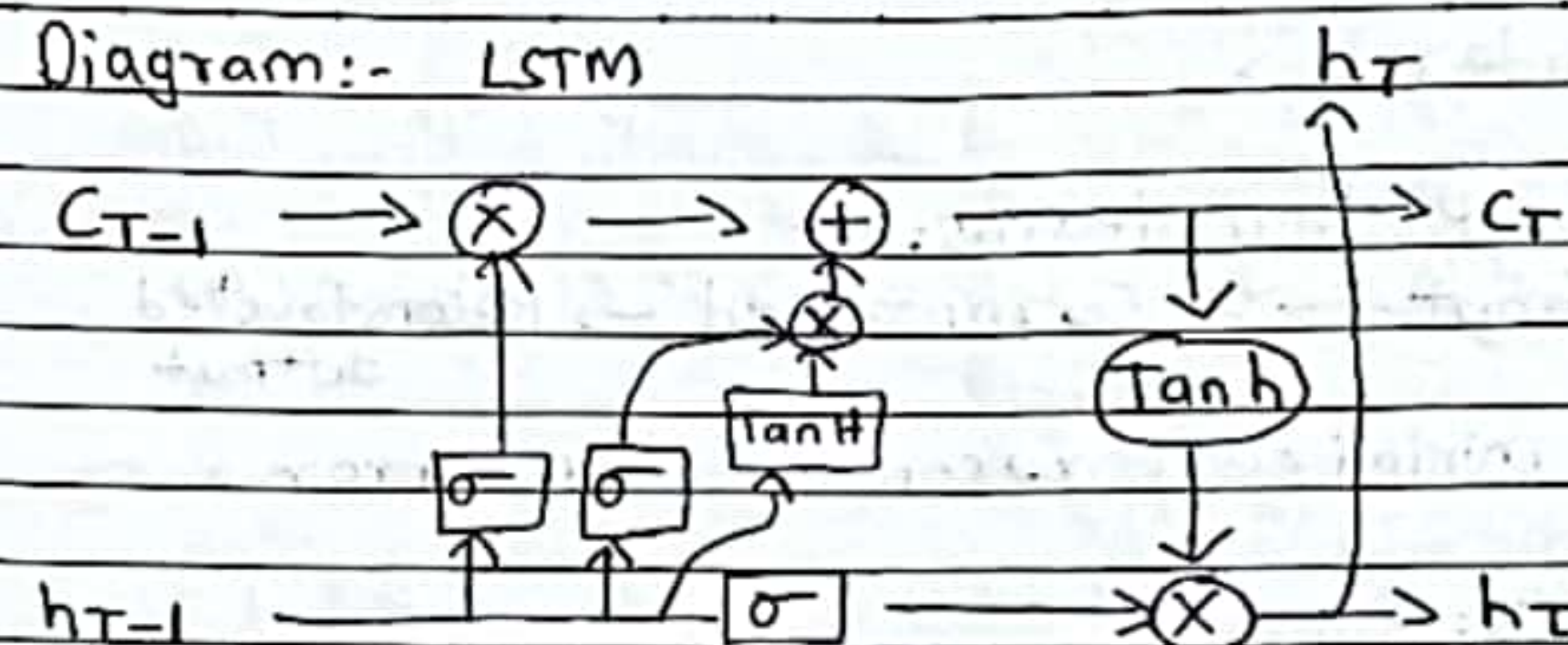
$$\tilde{h}_t = \tanh(w[h_t - h_{t-1}, x_t] + b)$$

4. final hidden state

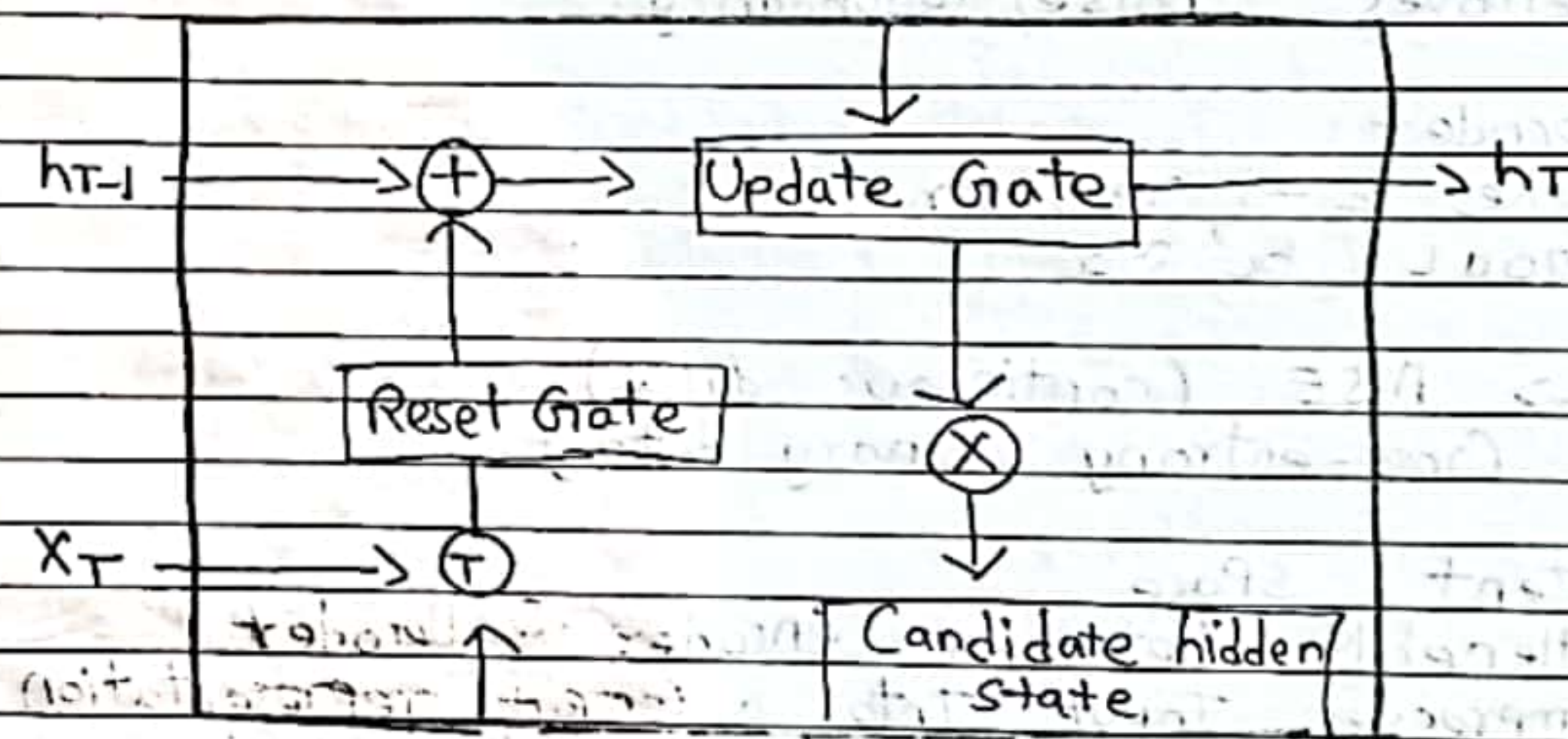
$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$$

#	LSTM	GRU
Memory	cell state + Hidden	Hidden
Gates	3 (forget, τ , o)	2 (update, reset)
Training Complexity	Slower higher	faster lower

Diagram:- LSTM



GRU



$+$ $\rightarrow$ Combines Memory

$\times$ $\rightarrow$ Gate filtering info

$t \rightarrow$ time Step

Autoencoder :->

Neural network that learns :->

Input $\rightarrow$ Compressed $\rightarrow$ Reconstructed output

Objective :- minimize reconstruction error

** Components :->

1. Encoder

- Input $x \rightarrow$ Compressed Vector z
- learns Important features
- Removes noise/redundancy.

2. Decoder

- Takes $z \rightarrow$ reconstructs x'
- Goal $x' \approx x$

Loss $\rightarrow$ MSE (continuous data)

Cross-entropy (binary data)

Latent Space

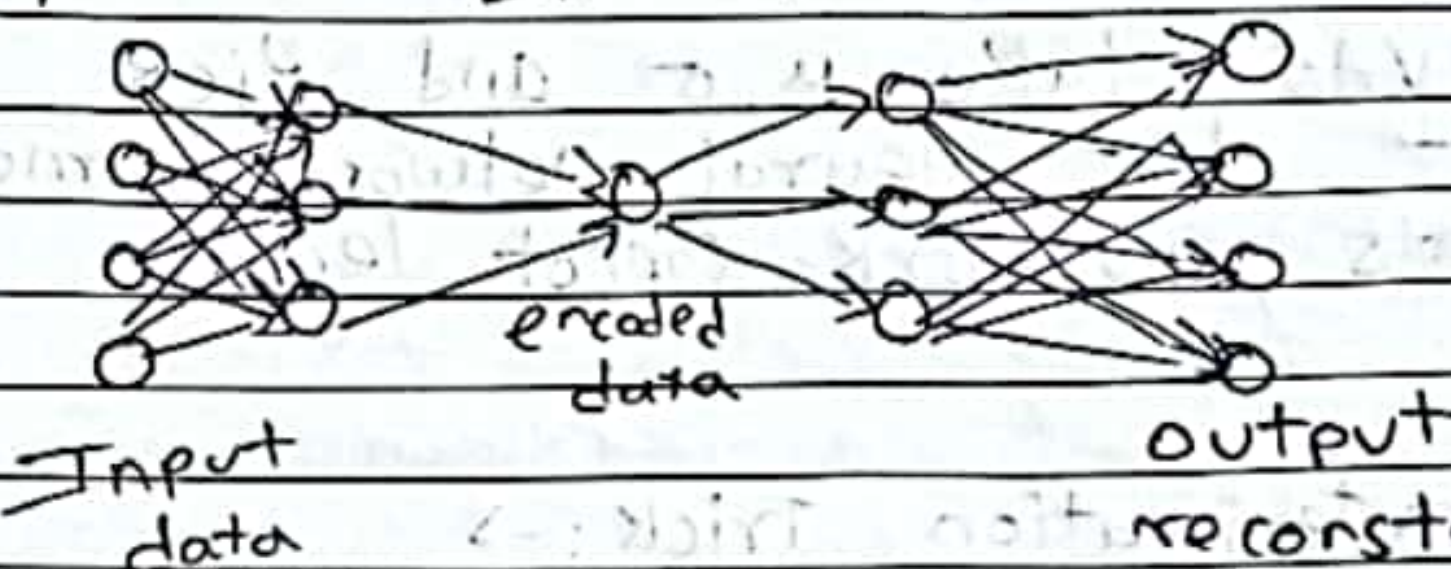
- $\rightarrow$ Bottleneck between encoder & Decoder
- $\rightarrow$ Compresses Input into a compact representation
- $\rightarrow$ Prevents memorization & learns dense features.

Cons: $\rightarrow$ Overfitting

$\Rightarrow$ Non Generative

$\Rightarrow$ Discontinuous latent space

Encoder Latent space Decoder



Variational Autoencoders :->

$\rightarrow$ "latent space is regularized to follow the normal distribution"

Normal Autoencoder $\rightarrow$ one dot (fixed point)

VAE $\rightarrow$ map probabilistic distribution (not point but cloud)

μ (mean) $\rightarrow$ center of cloud

σ (SD) $\rightarrow$ wide/spread of cloud

This cloud solves :-

- ① Close points $\rightarrow$ similar objects
- ② Moving in space $\rightarrow$ smooth change
- ③ Create new data

** Key components :->

① Probabilistic Encoder :-> Point Probability Around here.

② Latent Sampling: $\rightarrow$ Pick a Point Randomly

③ Probabilistic Decoder: $\rightarrow$ Pick the Point & Turns Into valid output.

* Problem: In VAE take μ, σ and Pick random point but neural network learns using gradients, Network cannot learn randomness.

* Fix: Reparameterization Trick: $\rightarrow$ Instead of Sampling Pick directly from the distribution.

1. Sample ϵ from a normal distribution (mean 0, std deviation 1)

2. Then Compute $z = \mu + (\sigma \times \epsilon)$, randomness number

Same Input $\rightarrow$ Same μ, σ

Different $\epsilon \rightarrow$ Different z

$\rightarrow$ Model Learn differently

* VAE objectives: -

① Reconstruction loss: - measure using MSE, cross entropy

② KL-divergence: $\rightarrow$ Keep latent space close to normal distribution.

$$L = E[\log P(x/z)] - D_{KL}(q(z/x) || P(z))$$

Reconstruction Regularization

* Application: -

① Image generation

② Anomaly detection

③ Drug Discovery.

GPT-LLM

Generative $\rightarrow$ Generates new tokens Based on a learned Probability distribution.

Pre-Trained $\rightarrow$ first Trained on massive unsupervised dataset to learn general linguistic Patterns.

Transformer $\rightarrow$ Uses Attention mechanism to Process Input sequences in Parallel, capturing long-range dependencies.

* Decoder only Architecture.

GPT uses stacked-Decoder only setup.

• Masked Self-Attention

$\rightarrow$ Model cannot peek at future tokens.

• Auto regressive

$\rightarrow$ Predicts next token based on previous one

• Positional encoding

$\rightarrow$ Injects sequence order Information.

* Self-Attention - (Query, Key, Value) - every token is mapped to 3 vectors: $\rightarrow$

Q: What am I looking for

K: What Info do I have

V IF matched, what should I provide.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_K}}\right) \cdot V$$

$\rightarrow$ stabilizes gradients.

It weights Importance of each token relative to current Query token

* Phase 1 - Generative Pre-Training

• Self-Supervised Learning

- Trained on terabytes of raw text.
- Model becomes a strong foundation before any task-specific tuning.

Objective:- Maximize likelihood of correctly predicting the actual next word in a sequence.

Result: Model learns syntax, facts, reasoning and logic from raw patterns.

* Phase 2 - Alignment via RLHF

RLHF = Reinforcement Learning Human feedback.

Goal: To make model helpful, honest & harmless.

Pipeline:-

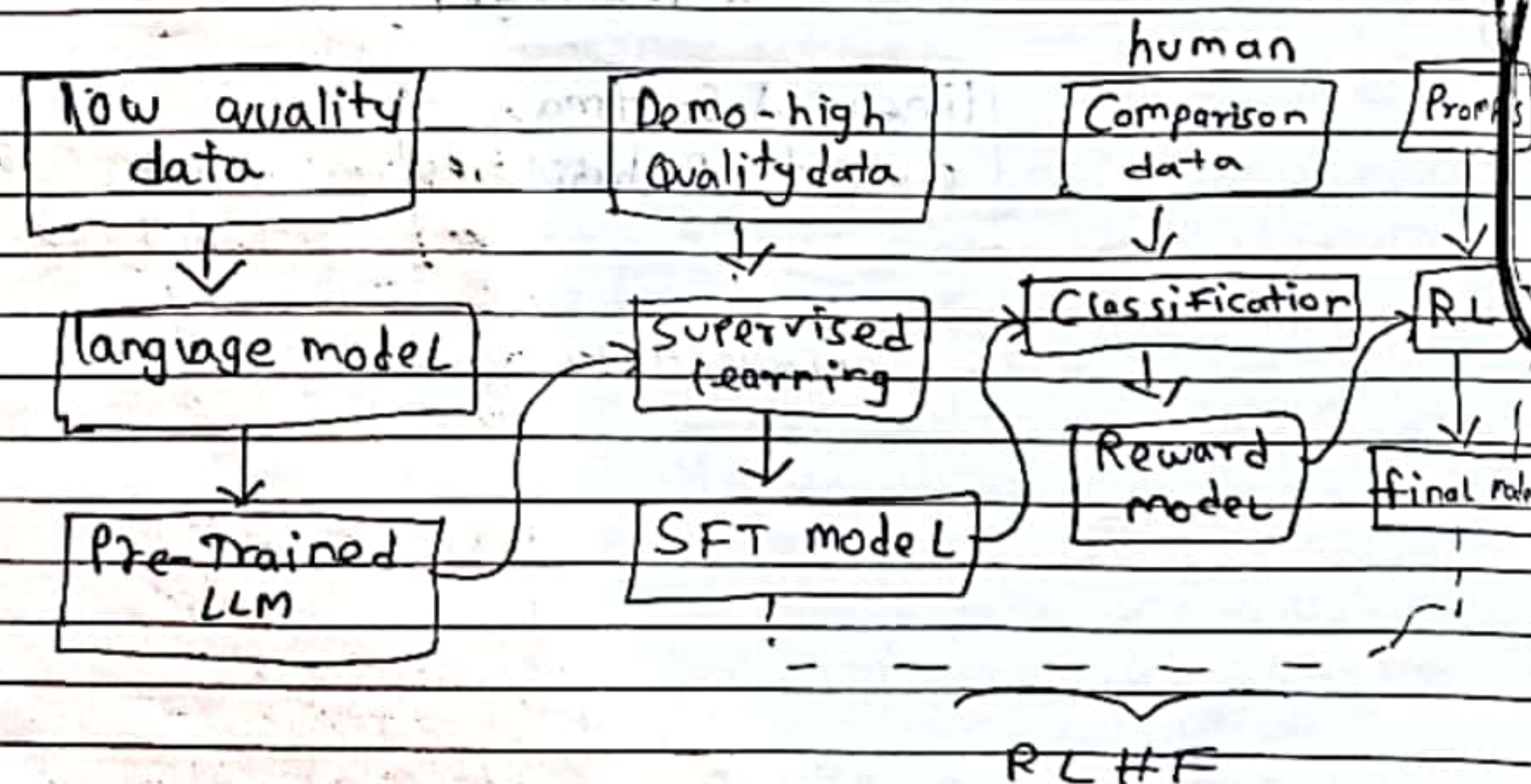
- ① Supervised fine tuning:-
fine tune the pretrained LLM on high-quality human-written demonstration data.
- ② Reward model Training:-
→ human ranks multiple model outputs for the same prompt.
→ Train a separate reward model that predicts a quality score.
- ③ Use Proximal Policy optimization to update GPT Policy to generate responses that maximises reward model scores.

* In-Context Learning:-

→ LLMs can perform tasks without any weight updates - just by seeing example in the Prompt.

- Zero-Shot Prompting → No examples given
- Few-Shot → 2-3 Examples before the final Query model infers the pattern.
- Chain of Thought reasoning → Prompting the model to think step by step which improves complex reasoning.

Cons: → Hallucinations (confident but false info)
→ Knowledge cutoff
→ Bias due to training data
→ Environmental cost.



Input Tokens
+ Positional Encoding

$N \times$ Masked Multi-head
Self-attention

Add & Norm

Feed forward Network

Add & Norm

$\times N \times$ layers

Linear & Softmax
Output Probabilities

Next token

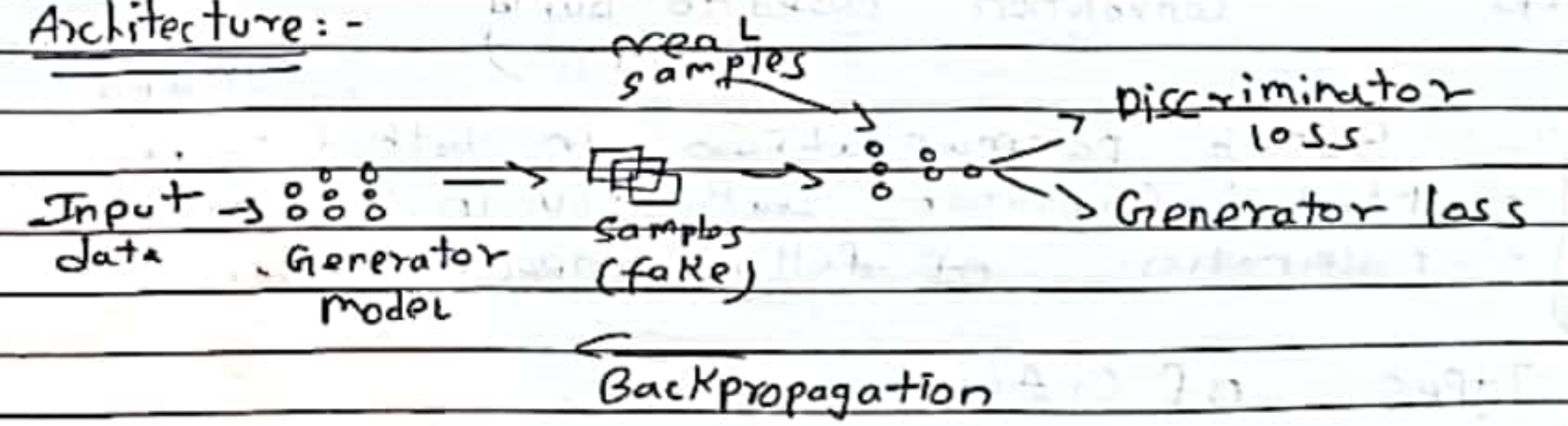
residual
skip

Generative Adversarial Networks: $\rightarrow$
Neural networks learns to generate new data by using 2 models against each other

① Generative model: $\rightarrow$ Distribution of data to create new similar instances
ex: GANs, VAE
It takes random noise, fake data looks real.

② Discriminator (Judge): $\rightarrow$ Takes input decides real or fake
ex: Image Classification.

Architecture: -



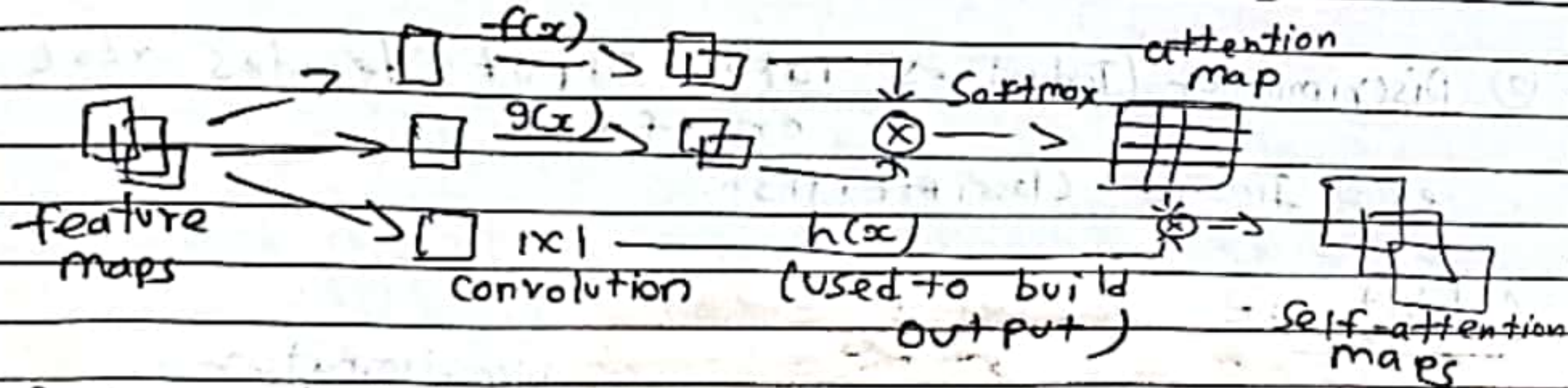
$\rightarrow$ Model improves till its good enough to create fake samples to fool discriminator through backpropagation.

- Process: $\rightarrow$
- ① Sampling: Start with noise, create fake data
 - ② Train discriminator: real Img $\rightarrow 1$
fake Img $\rightarrow 0$
 - ③ Train generator: freeze discriminator
Train generator.
 - ④ Compute loss: If 0 says fake $\rightarrow$ Bad Training
If 0 says real $\rightarrow$ Good Training

⑤ Equilibrium: After round G & D becomes Strong.

Deep Convolution Generative Adversarial Network:- (DCGAN's)

-> GAN designed for Images (Instead of basic NN uses Convolution layers)



Pros:-
-> Batch normalization in both networks
-> RELU in Generator, Leaky RELU in discriminator
-> Elimination of fully connected layers.

Types of GAN:-

① CGAN: Conditional GANs allows for targeted generation by providing extra info to both G & D.

② CycleGAN: Translate Images from one domain to another.
ex. Horse to zebra.

③ StyleGAN: Control over various style of generated images like hair, color, age, pose.

Challenges & solutions:

1. Mode Collapse:-

G produces only few types of samples, thus ignoring diversity.

Sol:-> Unrolled GANs, minibatch discrimination.

2. Nash equilibrium:-

G & D fail to reach a stable balance

Sol:-> WGAN loss

3. Vanishing gradients:-

D becomes too strong, G receives no learning signal

Sol:- Label smoothing

Applications:-

-> Face synthesis
-> Healthcare.

Diffusion Models $\rightarrow$

Goal $\rightarrow$ Generate realistic images from random noise.

Key trick $\rightarrow$ Train network to undo-noise, step by step.

Clean Image x_0 $\rightarrow$ Slight noise x_1 $\rightarrow$ More noise x_t $\rightarrow$ Pure noise x_T $\xleftarrow{\text{reverse (learned)}}$

$\rightarrow$ forward (add noise, fixed formula, not learned)

* Forward process: (fixed, q)

$\rightarrow$ Add a little Gaussian noise at each step using schedule $\beta_1, \beta_2, \dots, \beta_T$

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon, \quad \epsilon \sim N(0, 1)$$

* shortcut $\rightarrow (x_0 - x_T)$ in one step

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \text{where } \bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i)$$

ex: $T = 1000$, β from $0.0001 \rightarrow 0.02$

$\Rightarrow$ At $T = 200 \rightarrow$ slightly blurry

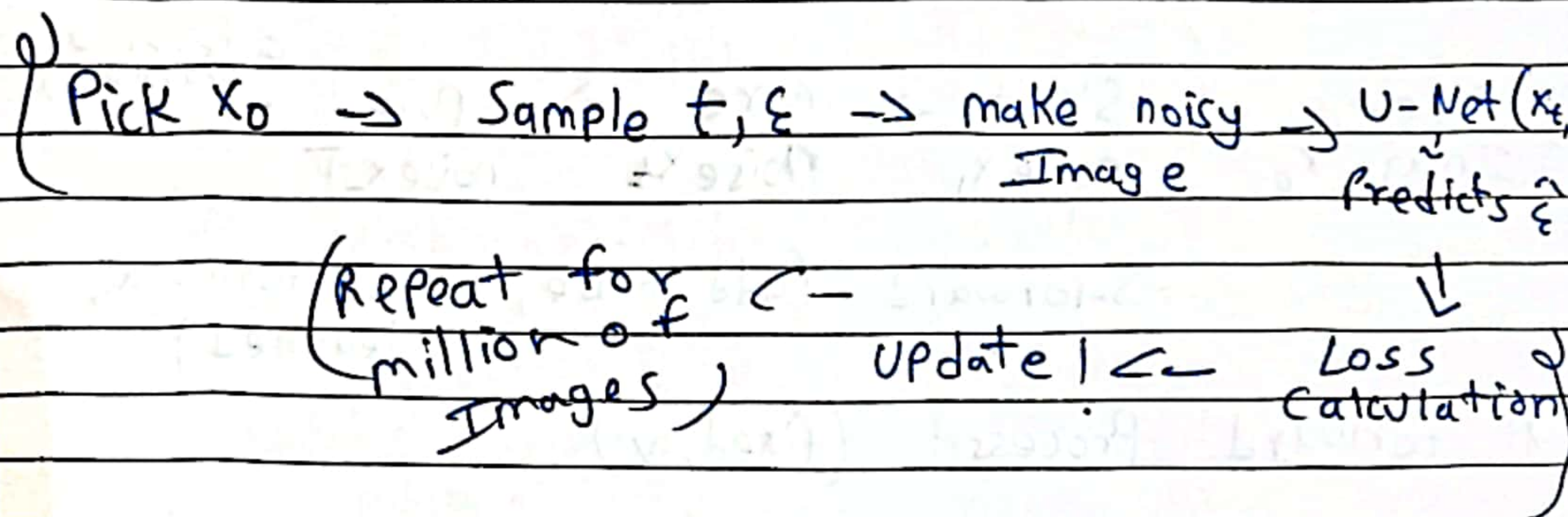
At $T = 800 \rightarrow$ Unrecognisable

$T = 1000 \rightarrow$ Pure noise

* Training

→ Train a U-Net (noise predictor $\hat{\epsilon}$) to Predict which noise was added to x_t .

$$\text{Loss} = |\epsilon - \hat{\epsilon}(x_t, t)|^2 \quad \text{MSE on noise!}$$



* Inference (Generating new image)

→ Start from pure noise x_T denoise T -times to get x_0

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t \cdot [x_t - \beta_t \cdot \frac{1 - \bar{\alpha}_t}{1 - \bar{\alpha}_t} \cdot \hat{\epsilon}(x_t, t)]}} + \sigma_t \cdot z$$

$z \sim N(0, 1)$

* Condⁿ generation (text → Image)

→ Inject text prompt into U-Net via cross-attention.

$$\hat{\epsilon} = \text{U-Net}(x_t, t, \text{"a running cat"})$$

Text encoded by CLIP → embedding → Cross-Attention layers in U-Net

*

VAE vs Diffusion

encoder

↓

single z

↓

decoder

Fast

sometimes blurry

T-step noise

↓

T-step Denoise

Slower

Much high Quality

—

λ -

Self-Attention & Transformers

M	T	W	T	F	S	S
Page No.:						YOUVA
Date:						

- * Fully connected layers can only look at a fixed window $\rightarrow$ can't capture long-range context.
- * RNN $\rightarrow$ forgets early tokens.

Self-attention: each token looks at all other tokens simultaneously.

* Working: $\rightarrow$ Q, K, V

Each Input Vector a_i produces 3 vectors via learnable weight matrices:

$$\begin{aligned} q_i &= W_q \cdot a_i \quad (\text{Query - what am I looking for}) \\ k_i &= W_k \cdot a_i \quad (\text{Key - what do I contain}) \\ v_i &= W_v \cdot a_i \quad (\text{value - what info do I pass on?}) \end{aligned}$$

① Attention-Score (dot product of query with all the keys):

$$a_{1,i} = q_i \cdot k_i \leftarrow \text{how relevant is token } i \text{ to token } 1?$$

② Softmax to get weights that sum to 1.

$$\tilde{a}_{1,i} = \text{Softmax} (a_{1,i} / \sqrt{d_k})$$

$\sqrt{d_k} \rightarrow$ normalization gradients

③ $p_1 = \sum_i \tilde{a}_{1,i} \cdot v_i \leftarrow$ output for token 1

contains global context.

* Matrix Form

- efficient - whole sequence at once.
- Fully Parallelised
- All outputs computed in one shot.

* Multi-head Self Attention

- One set (Q, K, V) captures one type of relevance.

Use H heads for H types:

Each Head i has its own $W_{Q_i}, W_{K_i}, W_{V_i}$ → output J_i .

Concat $(b_1, b_2, \dots, b_H)$ → linear layer → final output

* Positional encoding

- self-attention is order-agnostic
- shuffling tokens gives same result.

Fix: add a position vector e_i to each input before attention:

Input to attention = $a_i + e_i$

e_i from cosine similarity.

Note: CNN c Self-attention:

CNN = Self attention with fixed local window.

* Output types:

- (a) $N \rightarrow N$ (same length) $\rightarrow$ Parts of speech
Each token $\rightarrow$ label L (POS tagging)
- (b) $N \rightarrow 1$ (one label L)
Whole sequence $\rightarrow$ class (Sentiment)
- (c) $N \rightarrow N'$ (seq2seq)
model decides length (Translator)

* Applications:

1. Speech (Truncated self-attention)
2. Images (DETR)
3. Text: (BERT, GPT)

Transformer: Encoder & Decoder:

1. Seq2Seq

Input = Sequence of any length
output = Sequence of model-determined length

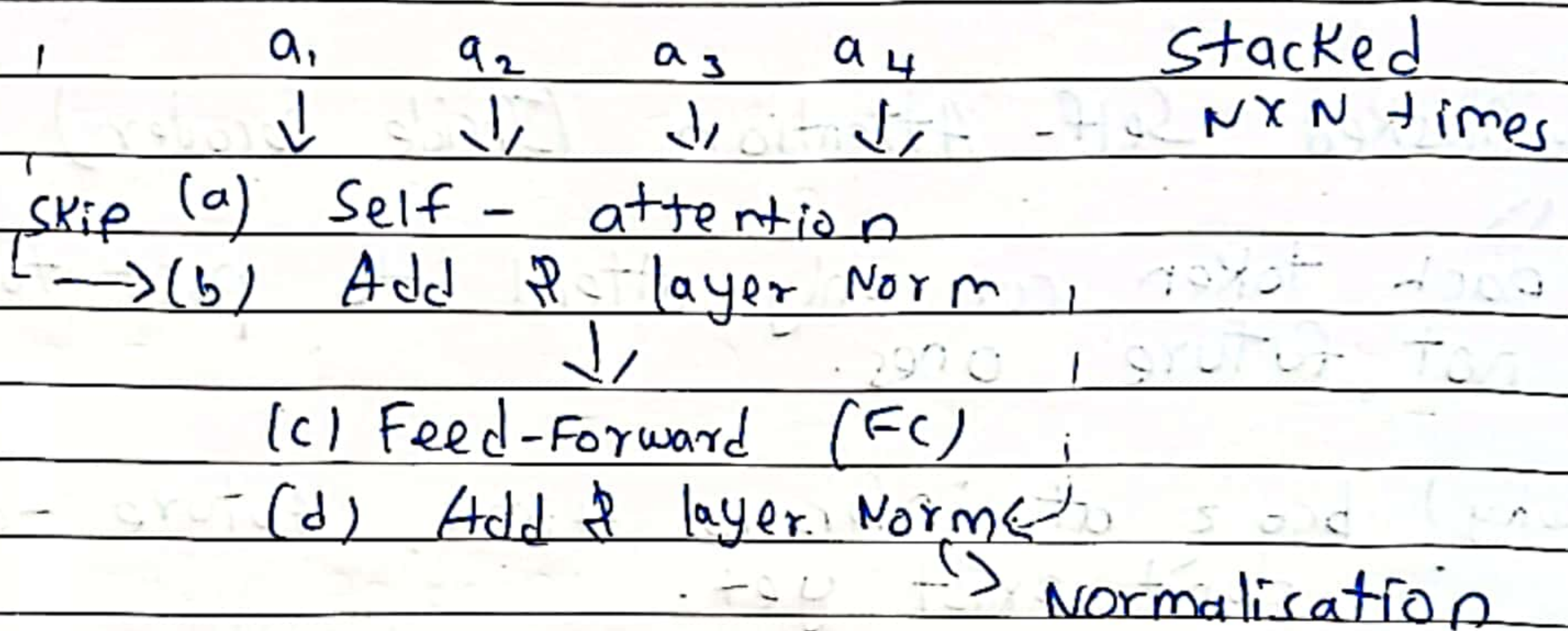
uses: Speech recogⁿ, Chatbot, Q & A

Input sequence $\rightarrow$ Encoder $\rightarrow$ Decoder $\rightarrow$ Output sequence

Note: Syntactic Parsing $\rightarrow$ Serialise the tree as a string
Multi-label classification
Object Detection (DETR)

2. Encoder - Transformer Block

- (a) Self-attention $\rightarrow$ each token attends to all other
- (b) Add $\&$ Norm (Residual Connection + layer norm)
- (c) Feed-Forward (Fc)
 - $\rightarrow$ applied Independently to each token
- (d) Add $\&$ Norm again



Residual = Input + Block output

$\rightarrow$ Prevents vanishing gradients

3. Decoder - Autoregressive (AT)

$\rightarrow$ Generates output one token at a time, feeding each output back as next Input.

Ex: START $\rightarrow$ "I" $\rightarrow$ "I love" $\rightarrow$ "I love deep" $\rightarrow$...

END

START $\rightarrow$ Decoder $\rightarrow$ I $\rightarrow$ Decoder $\rightarrow$ love $\rightarrow$... End

Feed back as Input.

Note: $\rightarrow$ only stops when it outputs the special END token, otherwise not stops.

$\rightarrow$ ERROR accumulation risk:

One wrong token $\rightarrow$ All future tokens affected.

4. Masked Self-Attention (Inside Decoder)

$\hookrightarrow$ each token can only attend to past tokens, not future ones.

why? bcoz at inference time, future tokens don't exist yet.

(Self-attention $\rightarrow$ encoder)
(every token $\rightarrow$ each other)

5. Cross-Attention (How encoder talks to Decoder)

$\rightarrow$ Bridge betwⁿ encoder & Decoder

$\rightarrow$ Decoder's output is the Query:

$\rightarrow$ encoder's output provides Key & Values.

Encoder (K & V) $\rightarrow$ Cross-Attention $\rightarrow$ Context-aware
Decoder (Q) $\rightarrow$ Q.KT $\rightarrow$ softmax $\rightarrow$ V (next token output)

6. Full Transformer Block.

- (A) Masked Self-Attention
- (B) Add γ layer Norm
- (C) Cross-Attention
- (D) Add γ layer Norm
- (E) Feed-Forward + Add γ layer Norm